# Recurrent Neural Networks (RNNs) - Notes

## 1. Introduction to RNNs

Recurrent Neural Networks (RNNs) are a type of neural network designed to handle sequential data where order and context matter. Unlike traditional feedforward networks, RNNs have loops that allow information to persist across time steps. This makes them ideal for tasks like time series forecasting, speech recognition, and text generation.

## 2. RNN Architecture

An RNN processes an input sequence one element at a time while maintaining a 'hidden state' that carries information about previous inputs. At each time step, the RNN takes two inputs — the current input ($x_t$) and the previous hidden state ($h_{t-1}$) — and produces a new hidden state ($h_t$) and possibly an output ($y_t$).

Mathematically:

$$h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b)$$
$$y_t = W_y * h_t + b_y$$

WHAT IS TANH? ➔**"Tanh is a zero-centered activation function that maps input values between -1 and 1, making it better than sigmoid for hidden layers."**

## 3. Key Terms in RNNs

- Hidden State ($h_t$): Memory of the network that stores past information.
- Sequence: Ordered data points, e.g., words in a sentence or daily stock prices.
- Time Step: Each position in a sequence that the RNN processes.
- Backpropagation Through Time (BPTT): The training method used to compute gradients over time steps.
- Unrolling: Visual representation of an RNN across time steps to show how it processes sequences.

## 4. Problems in Training RNNs

During training, RNNs use Backpropagation Through Time (BPTT) to update weights. However, this process can lead to two major problems:

1. Vanishing Gradient Problem:

When gradients become very small during backpropagation, earlier layers receive minimal updates. As a result, the network fails to learn long-term dependencies.

2. Exploding Gradient Problem:

When gradients grow exponentially, the weight updates become too large, causing instability and poor convergence.

Solutions:

- Use gated architectures like LSTM or GRU.
- Apply gradient clipping.
- Use smaller learning rates.
- Employ proper weight initialization techniques.

## 5. Applications of RNNs

RNNs are used in many real-world applications that involve sequential or time-dependent data:

- Text Generation: Generate new sentences based on learned patterns.
- Machine Translation: Translate text from one language to another.
- Sentiment Analysis: Determine emotion from text.
- Time Series Forecasting: Predict future values like stock prices or weather.
- Speech Recognition: Convert spoken language into text.

## 6. Summary

Recurrent Neural Networks are powerful for modeling sequential data but suffer from training challenges like exploding and vanishing gradients. Modern variants such as LSTM and GRU effectively overcome these issues, making them widely used in NLP and time-series applications.